



ESPE
UNIVERSIDAD DE LAS FUERZAS ARMADAS
INNOVACIÓN PARA LA EXCELENCIA



Departamento: Ciencias de la computación

Carrera: Ing. Electrónica y Automatización

Asignatura: Fundamentos de la programación

NRC: 28561

Apellidos y nombres del estudiante:

Iza Flores Esteban Daniel

Guía de Ejercicios de Vecto y MTZ

Ejercicio 2.1.1. Vector al revés

Enunciado. Desarrolle un programa que lea las N componentes de un vector y las escriba a continuación en orden inverso al introducido. El valor de N se introduce por teclado y se supone que será igual o menor a 10.

Idea clave. Leer el vector en orden normal y luego recorrerlo desde la última posición hasta la primera.

Objeto	Nombre	Valor	Tipo
Vector de 10 componentes	vec	var	real
Contador	i	var	entero
Uno	1	cte	entero
Número de componentes del vector	N	var	entero
Aspecto de resolución		Orientación para el estudiante	
Entrada		Identifica qué datos se leen por teclado y cuáles ya están definidos.	
Proceso		Describe el recorrido necesario con ciclos y las condiciones o acumulaciones requeridas.	
Salida		Especifica qué se debe mostrar y en qué orden.	
Pensamiento crítico		¿Por qué el segundo recorrido debe decrementar el índice? ¿Qué pasaría si se imprime nuevamente de 1 a N?	

Análisis

El ejercicio pide que se lea un vector de tamaño N (≤ 10) e imprimirlo en orden inverso.

Algoritmo (pseudocódigo)

Proceso MostrarVectorInvertido

Definir numeros Como Real

Definir cantidad, j Como Entero

Dimension numeros[10]

Escribir "Ingrese la cantidad de elementos:"

Leer cantidad

Para j <- 0 Hasta cantidad - 1 Hacer

 Escribir "Ingrese un numero:"

 Leer numeros[j]

FinPara

Escribir "Elementos del vector en orden inverso:"

Para j <- cantidad - 1 Hasta 0 Con Paso -1 Hacer

 Escribir numeros[j]

FinPara

FinProceso

Tabla de objetos:

Objeto / Variable	Tipo de Dato	Descripción
numeros	Real	Vector donde se almacenan los números ingresados
cantidad	Entero	Cantidad de elementos que tendrá el vector
j	Entero	Variable de control para los ciclos
numeros[10]	Real	Arreglo con capacidad máxima de 10 elementos

Prueba de Escritorio

Paso	j	numeros[j]	Salida
Ingresa cantidad	-	5	-
Leer dato	0	10	-
Leer dato	1	20	-
Leer dato	2	30	-

Paso	j	numeros[j]	Salida
Leer dato	3	40	-
Leer dato	4	50	-
Mostrar	4	50	50
Mostrar	3	40	40
Mostrar	2	30	30
Mostrar	1	20	20
Mostrar	0	10	10

Código en C

```
#include <stdio.h>

int main() {
    int cantidad, j;
    float numeros[10];
    printf("Ingrese la cantidad de elementos: ");
    scanf("%d", &cantidad);
    for(j = 0; j < cantidad; j++) {
        printf("Ingrese un numero: ");
        scanf("%f", &numeros[j]);
    }

    // Mostrar vector invertido
    printf("\nElementos en orden inverso:\n");

    for(j = cantidad - 1; j >= 0; j--) {
        printf("%.2f\n", numeros[j]);
    }
}
```

```

    }

    return 0;
}

```

```

C:\Users\User\Downloads\Un... X + v
Ingrese la cantidad de elementos: 5
Ingrese un numero: 1
Ingrese un numero: 2
Ingrese un numero: 3
Ingrese un numero: 2
Ingrese un numero: 3

Elementos en orden inverso:
3.00
2.00
3.00
2.00
1.00

Process returned 0 (0x0)   execution time : 5.267 s
Press any key to continue.

```

Reflexión

Es importante recorrer el vector desde el final hacia el inicio porque el ejercicio pide mostrar los elementos en el orden opuesto al que se ingresaron. Además, esta práctica permite entender que la forma de recorrer un vector influye directamente en el resultado obtenido.

Ejercicio 2.1.2. Norma de un vector al cuadrado

Enunciado. Desarrolle un programa que lea las n componentes reales de un vector vec y calcule la norma al cuadrado: $m = \text{vec}^T \times \text{vec} = \text{suma de } \text{vec}(i)^2$. El tamaño n se introduce por teclado y puede suponerse igual o menor que 10.

Idea clave. Inicializar res en 0 y acumular el cuadrado de cada componente leída.

Objeto	Nombre	Valor	Tipo
Vector de 10 componentes	vec	var	real
Contador	i	var	entero
Número de componentes del vector	n	var	entero
Norma al cuadrado	res	var	real
Uno	1	cte	entero
Aspecto de resolución		Orientación para el estudiante	

Entrada		Identifica qué datos se leen por teclado y cuáles ya están definidos.	
Proceso		Describe el recorrido necesario con ciclos y las condiciones o acumulaciones requeridas.	
Objeto	Nombre	Valor	Tipo
Salida		Especifica qué se debe mostrar y en qué orden.	
Pensamiento crítico		¿Por qué crees debe iniciar en cero? ¿Qué error aparecería si se usa un valor basura no inicializado?	

Análisis

Suma los cuadrados de los elementos

Algoritmo (pseudocódigo)

Proceso CalcularNormaVector

Definir datos Como Real

Definir cantidad, k Como Entero

Definir suma Como Real

Dimension datos[10]

Escribir "Ingrese la cantidad de elementos:"

Leer cantidad

suma <- 0

Para k <- 0 Hasta cantidad - 1 Hacer

Escribir "Ingrese un numero:"

Leer datos[k]

suma <- suma + datos[k] * datos[k]

FinPara

Escribir "La suma de cuadrados es: ", suma

FinProceso

Tabla de objetos:

Objeto	Nombre	Valor	Tipo
Vector de números	datos	var	real
Cantidad de elementos	cantidad	var	entero
Contador	k	var	entero
Acumulador de suma	suma	var	real
Tamaño máximo del vector	10	cte	entero
Cero inicial	0	cte	entero

Prueba de Escritorio

Paso	k	datos[k]	Operación	suma
Inicial	-	-	suma = 0	0
Leer dato	0	2	$0 + (2 \times 2)$	4
Leer dato	1	3	$4 + (3 \times 3)$	13
Leer dato	2	4	$13 + (4 \times 4)$	29

Código en C

```
#include <stdio.h>

int main() {
    int cantidad, k;

    float datos[10];

    float suma = 0;

    printf("Ingrese la cantidad de elementos: ");

    scanf("%d", &cantidad);

    for(k = 0; k < cantidad; k++) {
        printf("Ingrese un numero: ");
```

```

scanf("%f", &datos[k]);

suma = suma + datos[k] * datos[k];

}

printf("\nLa suma de cuadrados es: %.2f\n", suma);

return 0;

}

```

```

C:\Users\User\Downloads\Un... x
Ingrese la cantidad de elementos: 8
Ingrese un numero: 4
Ingrese un numero: 4
Ingrese un numero: 5
Ingrese un numero: 5
Ingrese un numero: 5
Ingrese un numero: 2
Ingrese un numero: 2
Ingrese un numero: 6
La suma de cuadrados es: 151.00
Process returned 0 (0x0)   execution time : 7.260 s
Press any key to continue.

```

Reflexión

La variable `res` se inicializa en 0 ya que actúa como acumulador, permitiendo almacenar progresivamente la suma de los cuadrados de cada elemento del vector.

Ejercicio 2.1.3. Vector con término general dado

Enunciado. Sea la sucesión $v_k = k^2 + 3$. Desarrolle un programa que lea el número n de componentes que se quieren de la sucesión y las almacene en un vector `vec`, de forma que `vec(i) = v_i`. Puede asumir que n será menor o igual a 100.

Idea clave. Usar un ciclo para calcular cada término de la sucesión y almacenarlo en la posición correspondiente del vector.

Objeto	Nombre	Valor	Tipo
Vector de 100 componentes	<code>vec</code>	<code>var</code>	real
Contador	<code>i</code>	<code>var</code>	entero
Uno	1	<code>cte</code>	entero
Número de componentes del vector	<code>n</code>	<code>var</code>	entero
Aspecto de resolución		Orientación para el estudiante	
Entrada		Identifica qué datos se leen por teclado y cuáles ya están definidos.	
Proceso		Describe el recorrido necesario con ciclos y las condiciones o acumulaciones requeridas.	
Salida		Especifica qué se debe mostrar y en qué orden.	
Pensamiento crítico		¿Qué diferencia existe entre el índice usado en el enunciado y el índice que usa C cuando empieza en 0?	

Análisis:

Calcula cada término de la sucesión y lo almacena en la posición correspondiente del vector

Código Pseint :

Proceso SucesionVector

Definir vec Como Entero

Definir n, i Como Entero

Dimensión vec[100]

Escribir "Ingrese la cantidad de elementos:"

Leer n

Para i <- 0 Hasta n - 1 Hacer

vec[i] <- i * i + 3

FinPara

Escribir "Elementos de la sucesión:"

Para i <- 0 Hasta n - 1 Hacer

Escribir "vec[, i, "] = ", vec[i]

FinPara

FinProceso

Tabla de objetos:

Objeto	Nombre	Valor	Tipo
Vector de la sucesión	vec	var	entero
Cantidad de elementos	n	var	entero
Contador	i	var	entero
Tamaño máximo del vector	100	cte	entero
Cero inicial	0	cte	entero
Constante de la fórmula	3	cte	entero

Prueba de escritorio:

Paso i Cálculo $vec[i]$

$$1 \quad 0 (0^2 + 3) 3$$

$$2 \quad 1 (1^2 + 3) 4$$

$$3 \quad 2 (2^2 + 3) 7$$

$$4 \quad 3 (3^2 + 3) 12$$

$$5 \quad 4 (4^2 + 3) 19$$

Código en C:

```
#include <stdio.h>
```

```
int main() {
```

```
    int vec[100];
```

```
    int n, i;
```

```
    printf("Ingrese la cantidad de elementos: ");
```

```
    scanf("%d", &n);
```

```
    for(i = 0; i < n; i++) {
```

```
        vec[i] = i * i + 3;
```

```
    }
```

```
    printf("\nElementos de la sucesión:\n");
```

```
    for(i = 0; i < n; i++) {
```

```
        printf("vec[%d] = %d\n", i, vec[i]);
```

```
    }
```

```
    return 0;
```

```
}
```

```

Ingrese la cantidad de elementos: 5

Elementos de la sucesión:
vec[0] = 3
vec[1] = 4
vec[2] = 7
vec[3] = 12
vec[4] = 19

Process returned 0 (0x0)   execution time : 1.579 s
Press any key to continue.

```

Reflexión :

Este ejercicio demuestra la importancia de los vectores y los ciclos para almacenar y generar datos automáticamente, facilitando el trabajo con sucesiones matemáticas de manera ordenada y eficiente.

Ejercicio 2.1.4. Comprobar si dos valores pertenecen a un vector

Enunciado. Realice un algoritmo que lea dos números enteros por teclado y determine si ambos valores forman parte de un vector de enteros previamente definido de dimensión 15.

Idea clave. Usar dos variables bandera para marcar si num1 y num2 se encuentran en el vector.

Objeto	Nombre	Valor	Tipo
Número de componentes del vector	N	cte	entero
Vector de 15 números	vec	var	entero
Variable auxiliar para el primer dato	aux1	var	entero
Variable auxiliar para el segundo dato	aux2	var	entero
Contador	i	var	entero
Primer dato de entrada	num1	var	entero
Segundo dato de entrada	num2	var	entero
Cero	0	cte	entero
Uno	1	cte	entero
Aspecto de resolución		Orientación para el estudiante	
Entrada		Identifica qué datos se leen por teclado y cuáles ya están definidos.	
Proceso		Describe el recorrido necesario con ciclos y las condiciones o acumulaciones requeridas.	
Salida		Especifica qué se debe mostrar y en qué orden.	
Pensamiento crítico		¿Por qué se necesitan dos banderas y no una sola? ¿Qué condición final permite afirmar que ambos números están en el vector?	

Análisis:

El algoritmo recorre un vector de enteros para verificar si dos números ingresados por el usuario se encuentran ya almacenados

Código Pseint:

Proceso BuscarDosNumeros

Definir vec Como Entero

Definir aux1, aux2, i, num1, num2 Como Entero

Dimension vec[15]

vec[0] <- 5

vec[1] <- 8

vec[2] <- 12

vec[3] <- 20

vec[4] <- 7

vec[5] <- 15

vec[6] <- 3

vec[7] <- 18

vec[8] <- 25

vec[9] <- 10

vec[10] <- 30

vec[11] <- 2

vec[12] <- 14

vec[13] <- 9

vec[14] <- 6

aux1 <- 0

aux2 <- 0

Escribir "Ingrese el primer numero:"

Leer num1

Escribir "Ingrese el segundo numero:"

Leer num2 Para i <- 0 Hasta 14 Hacer

 Si vec[i] = num1 Entonces

 aux1 <- 1

 FinSi

 Si vec[i] = num2 Entonces

 aux2 <- 1

 FinSi

FinPara

Si aux1 = 1 Y aux2 = 1 Entonces

 Escribir "Ambos numeros pertenecen al vector"

SiNo

 Escribir "Uno o ambos numeros no pertenecen al vector"

FinSi

FinProceso

Tabla de objetos:

Objeto	Nombre	Valor	Tipo
Vector de números	vec	var	entero
Variable bandera del primer número	aux1	var	entero
Variable bandera del segundo número	aux2	var	entero
Contador	i	var	entero
Primer número ingresado	num1	var	entero
Segundo número ingresado	num2	var	entero

Objeto	Nombre	Valor	Tipo
Tamaño del vector	15	cte	entero
Cero inicial	0	cte	entero
Uno lógico	1	cte	entero

Prueba de escritorio:

i	¿vec[i] = num1?	aux1	¿vec[i] = num2?	aux2
0 5	No	0	No	0
1 8	No	0	No	0
2 12	Sí	1	No	0
3 20	No	1	No	0
4 7	No	1	No	0
5 15	No	1	No	0
6 3	No	1	No	0
7 25	No	1	Sí	1

Código en C:

```
#include <stdio.h>

int main() {

    int vec[15] = {5, 8, 12, 20, 7, 15, 3, 18, 25, 10, 30, 2, 14, 9, 6};

    int aux1 = 0, aux2 = 0;

    int i, num1, num2;
```

```

printf("Ingrese el primer numero: ");

scanf("%d", &num1);

printf("Ingrese el segundo numero: ");

scanf("%d", &num2);

for(i = 0; i < 15; i++) {

    if(vec[i] == num1) {

        aux1 = 1;

    }

    if(vec[i] == num2) {

        aux2 = 1;

    }

}

if(aux1 == 1 && aux2 == 1) {

    printf("\nAmbos numeros pertenecen al vector\n");

} else {

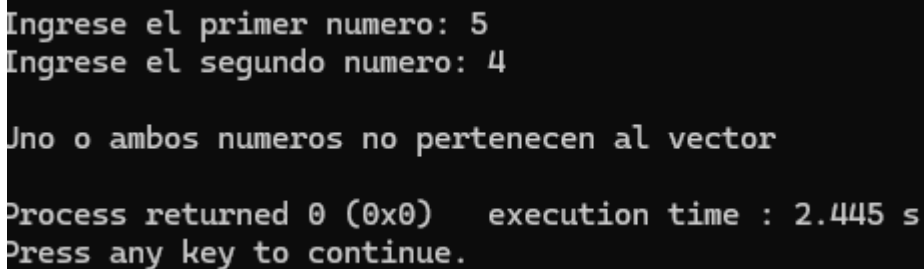
    printf("\nUno o ambos numeros no pertenecen al vector\n");

}

return 0;

}

```



```

Ingrese el primer numero: 5
Ingrese el segundo numero: 4

Uno o ambos numeros no pertenecen al vector

Process returned 0 (0x0)   execution time : 2.445 s
Press any key to continue.

```

Reflexión:

Este ejercicio demuestra cómo las variables bandera ayudan a controlar condiciones dentro de un recorrido, permitiendo verificar información de manera sencilla y eficiente.

Ejercicio 2.1.5. Vector de factoriales

Enunciado. Dado un vector Vec que contiene los primeros 15 números naturales, calcule un vector fact con sus factoriales y muéstrelolo por pantalla. Considere el problema de almacenamiento a partir de factoriales grandes.

Idea clave. Usar ciclos anidados: uno para recorrer los números y otro para calcular el factorial de cada componente.

Objeto	Nombre	Valor	Tipo
Vector de 15 números	Vec	var	entero
Vector de 15 números	fact	var	entero
Objeto	Nombre	Valor	Tipo
Contador	i	var	entero
Contador	j	var	entero
Cero	0	cte	entero
Uno	1	cte	entero
Quince	15	cte	entero
Aspecto de resolución		Orientación para el estudiante	
Entrada		Identifica qué datos se leen por teclado y cuáles ya están definidos.	
Proceso		Describe el recorrido necesario con ciclos y las condiciones o acumulaciones requeridas.	
Salida		Especifica qué se debe mostrar y en qué orden.	
Pensamiento crítico		¿Por qué el factorial crece tan rápido? ¿Qué tipo de dato conviene usar si el resultado supera la capacidad de int?	

Análisis: El algoritmo calcula el factorial de los primeros 15 números naturales utilizando ciclos anidados y almacena cada resultado en un vector.

Código en Pseint:

Proceso FactorialesVector

Definir Vec, fact Como Entero

Definir i, j Como Entero

Dimension Vec[15]

Dimension fact[15]

Para i <- 0 Hasta 14 Hacer

Vec[i] <- i + 1

FinPara

Para i <- 0 Hasta 14 Hacer

fact[i] <- 1

Para j <- 1 Hasta Vec[i] Hacer

fact[i] <- fact[i] * j

FinPara

FinPara

Escribir "Numero Factorial"

Para i <- 0 Hasta 14 Hacer

Escribir Vec[i], " ", fact[i]

FinPara

FinProceso

Tabla de objetos:

Objeto	Nombre	Valor	Tipo
Vector de números naturales	Vec	var	entero
Vector de factoriales	fact	var	entero
Contador para recorrer el vector i		var	entero
Contador para calcular factorial j		var	entero
Tamaño del vector	15	cte	entero
Valor inicial del factorial	1	cte	entero
Valor inicial del índice	0	cte	entero

Prueba de escritorio:

i	Vec[i]	Operación del factorial	fact[i]
0	1	1	1
1	2	1×2	2
2	3	$1 \times 2 \times 3$	6

i	Vec[i]	Operación del factorial	fact[i]
3	4	$1 \times 2 \times 3 \times 4$	24
4	5	$1 \times 2 \times 3 \times 4 \times 5$	120

Código en C:

```
#include <stdio.h>
```

```
int main() {
```

```
    int Vec[15];
```

```
    long long fact[15];
```

```
    int i, j;
```

```
    for(i = 0; i < 15; i++) {
```

```
        Vec[i] = i + 1;
```

```
    }
```

```
    for(i = 0; i < 15; i++) {
```

```
        fact[i] = 1;
```

```
        for(j = 1; j <= Vec[i]; j++) {
```

```

        fact[i] = fact[i] * j;

    }

}

printf("Numero    Factorial\n");

for(i = 0; i < 15; i++) {

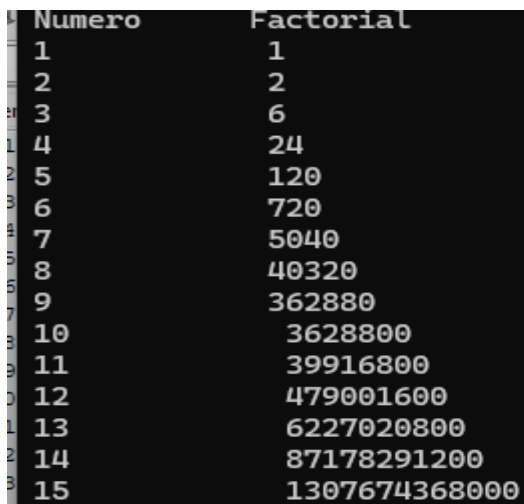
    printf("%d        %lld\n", Vec[i], fact[i]);

}

return 0;

}

```



Numero	Factorial
1	1
2	2
3	6
4	24
5	120
6	720
7	5040
8	40320
9	362880
10	3628800
11	39916800
12	479001600
13	6227020800
14	87178291200
15	1307674368000

Reflexión: Este ejercicio permite comprender cómo los ciclos anidados ayudan a resolver procesos repetitivos complejos y demuestra que los factoriales crecen muy rápidamente en tamaño.

Ejercicios de matrices

Ejercicio 2.2.1. Suma de componentes de una matriz

Enunciado. Halle la suma de las componentes de una matriz cuya dimensión y componentes se leen por teclado y por filas. Suponga que el número de filas N y columnas M son menores o iguales a 10.

Idea clave. Recorrer la matriz por filas con dos ciclos anidados y acumular cada componente en res.

Objeto	Nombre	Valor	Tipo
Matriz de 10x10 componentes	mat	var	real
Contador	i	var	entero
Contador	j	var	entero
Número de filas de la matriz	N	var	entero
Número de columnas de la matriz	M	var	entero
Suma de las componentes	res	var	real
Uno	1	cte	entero
Aspecto de resolución		Orientación para el estudiante	
Entrada		Identifica qué datos se leen por teclado y cuáles ya están definidos.	
Proceso		Describe el recorrido necesario con ciclos y las condiciones o acumulaciones requeridas.	
Salida		Especifica qué se debe mostrar y en qué orden.	
Pensamiento crítico		¿Por qué se necesita un ciclo para filas y otro para columnas? ¿Qué representa cada índice?	

Análisis:

El algoritmo recorre una matriz usando ciclos anidados para sumar todos sus elementos y mostrar el resultado final.

Código en Pseint:

Proceso SumaMatriz

Definir mat Como Real

Definir i, j, N, M Como Entero

Definir res Como Real

Dimension mat[10,10]

Escribir "Ingrese el numero de filas:"

Leer N

Escribir "Ingrese el numero de columnas:"

Leer M

res <- 0

Para i <- 0 Hasta N - 1 Hacer

Para j <- 0 Hasta M - 1 Hacer

Escribir "Ingrese un elemento:"

Leer mat[i,j]

res <- res + mat[i,j]

FinPara

FinPara

Escribir "La suma de los elementos es: ", res

FinProceso

Tabla de objetos:

Objeto	Nombre	Valor	Tipo
Matriz de números	mat	var	real
Contador de filas	i	var	entero
Contador de columnas	j	var	entero
Número de filas	N	var	entero
Número de columnas	M	var	entero
Acumulador de suma	res	var	real
Tamaño máximo de filas	10	cte	entero
Tamaño máximo de columnas	10	cte	entero
Valor inicial de la suma	0	cte	entero

Prueba de escritorio:

(matriz de 2x2) = [1,2;3,4]

i	j	mat [i,j]	res
---	---	--------------	-----

0	0	1	1
---	---	---	---

0	1	2	3
---	---	---	---

1	0	3	6
---	---	---	---

1	1	4	10
---	---	---	----

Código en C:

```
#include <stdio.h>
```

```
int main() {
```

```
    float mat[10][10];
```

```
    float res = 0;
```

```
    int i, j, N, M;
```

```
    printf("Ingrese el numero de filas: ");
```

```
    scanf("%d", &N);
```

```
    printf("Ingrese el numero de columnas: ");
```

```
    scanf("%d", &M);
```

```
    for(i = 0; i < N; i++) {
```

```
        for(j = 0; j < M; j++) {
```

```
            printf("Ingrese un elemento: ");
```

```
            scanf("%f", &mat[i][j]);
```

```
            res = res + mat[i][j];
```

```
        }
```

```

    }

    printf("\nLa suma de los elementos es: %.2f\n", res);

    return 0;
}

```

```

Ingrese el numero de filas: 2
Ingrese el numero de columnas: 2
Ingrese un elemento: 11
Ingrese un elemento: 12
Ingrese un elemento: 11
Ingrese un elemento: 22

La suma de los elementos es: 56.00

Process returned 0 (0x0)   execution time : 9.157 s
Press any key to continue.
|

```

Reflexión:

Este ejercicio ayuda a entender cómo funcionan las matrices y cómo los ciclos anidados permiten recorrer filas y columnas de manera ordenada y eficiente.

Ejercicio 2.2.2. Escritura de matriz en sentido inverso

Enunciado. Dada una matriz de $N \times N$ elementos, realice un algoritmo que recorra la matriz por filas desde la última a la primera y cada fila en sentido inverso, desde la última columna a la primera, mostrando cada elemento.

Idea clave. Inicializar los índices en la última posición y decrementar hasta llegar a la primera.

Objeto	Nombre	Valor	Tipo
Orden de la matriz	N	cte	entero
Matriz 4 x 4	mat	var	entero
Contador	i	var	entero
Contador	j	var	entero
Cero	0	cte	entero
Uno	1	cte	entero
Aspecto de resolución		Orientación para el estudiante	
Entrada		Identifica qué datos se leen por teclado y cuáles ya están definidos.	
Proceso		Describe el recorrido necesario con ciclos y las condiciones o acumulaciones requeridas.	
Salida		Especifica qué se debe mostrar y en qué orden.	
Pensamiento crítico		¿Qué cambia entre recorrer una matriz normalmente y recorrerla al revés? ¿Qué riesgo aparece con los límites de los índices?	

Análisis

El algoritmo recorre una matriz desde la última fila hasta la primera y muestra sus elementos en orden inverso utilizando ciclos descendentes.

código Pseint:

Proceso MatrizInvertida

Definir mat Como Entero

Definir i, j Como Entero

Definir N Como Entero

Dimension mat[4,4]

Escribir "Ingrese el orden de la matriz:"

Leer N

Para i <- 0 Hasta N - 1 Hacer

Para j <- 0 Hasta N - 1 Hacer

Escribir "Ingrese un elemento:"

Leer mat[i,j]

FinPara

FinPara

Escribir "Matriz en sentido inverso:"

Para i <- N - 1 Hasta 0 Con Paso -1 Hacer

Para j <- N - 1 Hasta 0 Con Paso -1 Hacer

Escribir mat[i,j]

FinPara

FinPara

FinProceso

Tabla de objetos:

Objeto / Variable	Tipo de Dato	Descripción
mat	Entero	Matriz donde se almacenan los elementos ingresados
mat[4,4]	Entero	Matriz bidimensional con capacidad máxima de 4x4

Objeto / Variable Tipo de Dato Descripción

i	Entero	Variable de control para recorrer las filas
j	Entero	Variable de control para recorrer las columnas
N	Entero	Orden de la matriz (número de filas y columnas)

Prueba de escritorio:

i j mat[i,j] Salida

1 1 4 4

1 0 3 3

0 1 2 2

0 0 1 1

código en c:

```
#include <stdio.h>
```

```
int main() {
```

```
    int mat[4][4];
```

```
    int i, j, N;
```

```
    printf("Ingrese el orden de la matriz: ");
```

```
    scanf("%d", &N);
```

```
    for(i = 0; i < N; i++) {
```

```
        for(j = 0; j < N; j++) {
```

```
            printf("Ingrese un elemento: ");
```

```
            scanf("%d", &mat[i][j]);
```

```
        }
```

```

    }

    printf("\nMatriz en sentido inverso:\n");

    for(i = N - 1; i >= 0; i--) {

        for(j = N - 1; j >= 0; j--) {

            printf("%d\n", mat[i][j]);

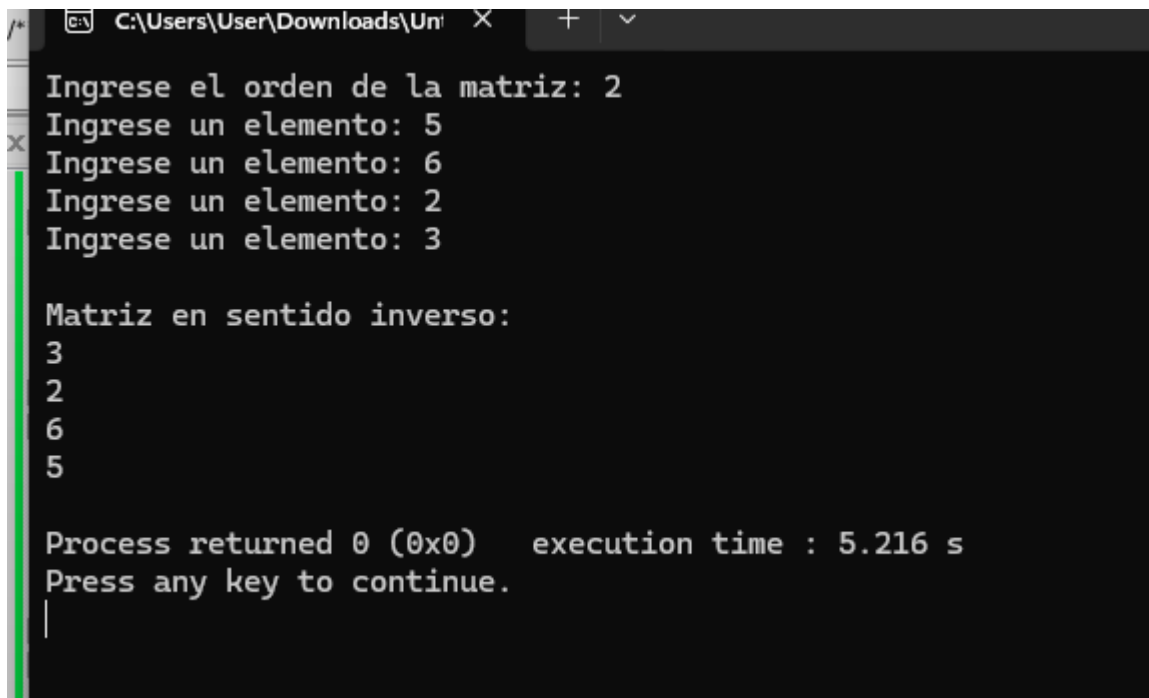
        }

    }

    return 0;

}

```



```

C:\Users\User\Downloads\Un... X + v
Ingrese el orden de la matriz: 2
Ingrese un elemento: 5
Ingrese un elemento: 6
Ingrese un elemento: 2
Ingrese un elemento: 3

Matriz en sentido inverso:
3
2
6
5

Process returned 0 (0x0)   execution time : 5.216 s
Press any key to continue.
|

```

Reflexión

Este ejercicio permite comprender cómo cambiar el sentido de recorrido de una matriz modifica completamente el orden de salida de los datos.

Ejercicio 2.2.3. Máximo de una fila

Enunciado. Escriba un programa que lea una matriz de N filas y N columnas de valores enteros. Luego debe pedir el número de una fila y mostrar por pantalla el valor de la mayor componente de esa fila.

Idea clave. Inicializar max con el primer elemento de la fila seleccionada y luego comparar con el resto de componentes de esa fila.

Objeto	Nombre	Valor	Tipo
Orden de la matriz	N	cte	entero
Matriz 4 x 4 componentes	mat	var	entero
Contador	i	var	entero
Contador	j	var	entero
Dato de entrada de fila	fil	var	entero
Máximo de la fila	max	var	entero
Uno y dos	1 y 2	cte	entero
Aspecto de resolución		Orientación para el estudiante	
Entrada		Identifica qué datos se leen por teclado y cuáles ya están definidos.	
Proceso		Describe el recorrido necesario con ciclos y las condiciones o acumulaciones requeridas.	
Salida		Especifica qué se debe mostrar y en qué orden.	
Pensamiento crítico		¿Por qué es mejor iniciar max con un valor de la propia matriz y no con un número fijo como -1000?	

Análisis

El algoritmo recorre una fila específica de una matriz para encontrar y mostrar el valor más grande almacenado en ella.

código Pseint:

Proceso MaximoFila

Definir mat Como Entero

Definir i, j, N Como Entero

Definir fil, max Como Entero

Dimension mat[4,4]

Escribir "Ingrese el orden de la matriz:"

Leer N

Para i <- 0 Hasta N - 1 Hacer

Para j <- 0 Hasta N - 1 Hacer

Escribir "Ingrese un elemento:"

Leer mat[i,j]

FinPara

FinPara

Escribir "Ingrese el numero de fila:"

Leer fil

max <- mat[fil,0]

Para j <- 1 Hasta N - 1 Hacer

Si mat[fil,j] > max Entonces

max <- mat[fil,j]

FinSi

FinPara

Escribir "El mayor valor de la fila es: ", max

FinProceso

Tabla de objetos:

Objeto	Nombre	Valor	Tipo
Orden de la matriz	N	var	entero
Matriz de números	mat	var	entero
Contador de filas	i	var	entero
Contador de columnas	j	var	entero
Fila seleccionada	fil	var	entero
Máximo de la fila	max	var	entero
Uno	1	cte	entero
Dos	2	cte	entero

Prueba de escritorio:

j mat[fil,j] max

Inicial 7 7

1 12 12

2 4 12

código en c:

```
#include <stdio.h>
```

```
int main() {
```

```
    int mat[4][4];
```

```
    int i, j;
```

```
    int N;
```

```
    int fil;
```

```
    int max;
```

```
    printf("Ingrese el orden de la matriz: ");
```

```
    scanf("%d", &N);
```

```
    for(i = 0; i < N; i++) {
```

```
        for(j = 0; j < N; j++) {
```

```
            printf("Ingrese el elemento [%d][%d]: ", i, j);
```

```
            scanf("%d", &mat[i][j]);
```

```
        }
```

```
    }
```

```
    printf("Ingrese la fila que desea analizar: ");
```

```
    scanf("%d", &fil);
```

```

max = mat[fil][0];

for(j = 1; j < N; j++) {

    if(mat[fil][j] > max) {

        max = mat[fil][j];

    }

}

printf("\nEl mayor valor de la fila %d es: %d\n", fil, max);

return 0;

}

```

```

Ingrese el orden de la matriz: 3
Ingrese un elemento: 5
Ingrese un elemento: 6
Ingrese un elemento: 5
Ingrese un elemento: 4
Ingrese un elemento: 6
Ingrese un elemento: 2
Ingrese un elemento: 5
Ingrese un elemento: 5
Ingrese un elemento: 1
Ingrese el numero de fila: 1

El mayor valor de la fila es: 6

Process returned 0 (0x0)   execution time : 8.533 s
Press any key to continue.

```

Ejercicio 2.2.4. Intercambiar las filas i, j de una matriz

Enunciado. Escriba un programa que intercambie las filas i y j de una matriz de enteros de N x N componentes, siendo i y j dos valores introducidos por teclado.

Idea clave. Usar una variable auxiliar para no perder información al intercambiar componente por componente.

Objeto	Nombre	Valor	Tipo
Orden de la matriz	N	cte	entero
Matriz de 4 x 4 componentes	mat	var	entero
Dato entrada primera fila	i	var	entero
Dato entrada segunda fila	j	var	entero
Contador	k	var	entero
Variable auxiliar	aux	var	entero
Uno	1	cte	entero

Aspecto de resolución	Orientación para el estudiante
Entrada	Identifica qué datos se leen por teclado y cuáles ya están definidos.
Proceso	Describe el recorrido necesario con ciclos y las condiciones o acumulaciones requeridas.
Salida	Especifica qué se debe mostrar y en qué orden.
Pensamiento crítico	¿Qué valor se perdería si se asigna directamente $mat[i][k] = mat[j][k]$ sin usar aux?

Análisis

El algoritmo intercambia dos filas de una matriz utilizando una variable auxiliar para conservar temporalmente los valores durante el cambio.

código Pseint:

Proceso IntercambiarFilas

Definir mat Como Entero

Definir i, j, k, N Como Entero

Definir aux Como Entero

Dimension mat[4,4]

Escribir "Ingrese el orden de la matriz:"

Leer N

Para i <- 0 Hasta N - 1 Hacer

Para j <- 0 Hasta N - 1 Hacer

Escribir "Ingrese un elemento:"

Leer mat[i,j]

FinPara

FinPara

Escribir "Ingrese la primera fila:"

Leer i

Escribir "Ingrese la segunda fila:"

Leer j

Para k <- 0 Hasta N - 1 Hacer

 aux <- mat[i,k]

 mat[i,k] <- mat[j,k]

 mat[j,k] <- aux

FinPara

Escribir "Matriz despues del intercambio:"

Para i <- 0 Hasta N - 1 Hacer

 Para j <- 0 Hasta N - 1 Hacer

 Escribir Sin Saltar mat[i,j], " "

 FinPara

 Escribir ""

FinPara

FinProceso

Tabla de objetos:

Objeto	Nombre	Valor	Tipo
Orden de la matriz	N	var	entero
Matriz de enteros	mat	var	entero
Primera fila a intercambiar	i	var	entero
Segunda fila a intercambiar	j	var	entero
Contador de columnas	k	var	entero
Variable auxiliar	aux	var	entero
Uno	1	cte	entero

Prueba de escritorio:

(Matrix de 2x2)

Supongamos:

i = 0

j = 1

Proceso de intercambio:

k aux mat[i,k] mat[j,k]

0 1 3 1

1 2 4 2

Matriz final

3 4

1 2

código en C:

```
#include <stdio.h>
```

```
int main() {
```

```
    int mat[4][4];
```

```
    int i, j, k, N;
```

```
    int aux;
```

```
    printf("Ingrese el orden de la matriz: ");
```

```
    scanf("%d", &N);
```

```
    for(i = 0; i < N; i++) {
```

```
        for(j = 0; j < N; j++) {
```

```
            printf("Ingrese el elemento [%d][%d]: ", i, j);
```

```
        scanf("%d", &mat[i][j]);

    }

}

printf("Ingrese la primera fila: ");

scanf("%d", &i);

printf("Ingrese la segunda fila: ");

scanf("%d", &j);

for(k = 0; k < N; k++) {

    aux = mat[i][k];

    mat[i][k] = mat[j][k];

    mat[j][k] = aux;

}

printf("\nMatriz despues del intercambio:\n");

for(i = 0; i < N; i++) {

    for(j = 0; j < N; j++) {

        printf("%d ", mat[i][j]);

    }

    printf("\n");

}

return 0;

}
```

```

C:\Users\User\Downloads\Un X + v
Ingrese el orden de la matriz: 2
Ingrese el elemento [0][0]: 1
Ingrese el elemento [0][1]: 2
Ingrese el elemento [1][0]: 1
Ingrese el elemento [1][1]: 3
Ingrese la primera fila: 1
Ingrese la segunda fila: 1

Matriz despues del intercambio:
1 2
1 3

Process returned 0 (0x0)   execution time : 10.386 s
Press any key to continue.

```

Reflexión:

Este ejercicio demuestra la importancia de las variables auxiliares para intercambiar datos sin perder información durante el proceso.

Ejercicio 2.2.5. Matriz traspuesta

Enunciado. Dada una matriz A de dimensión N x M, cuyas dimensiones reales y componentes se leen por teclado y por filas, calcule e imprima la matriz B de dimensión M x N, tal que $B = A^T$. Suponga que filas y columnas de A son menores o iguales a 10.

Idea clave. Al leer cada componente $A[i][j]$, almacenarla en $B[j][i]$. Luego imprimir B con dimensiones invertidas.

Objeto	Nombre	Valor	Tipo
Matriz de 10x10 componentes	A	var	real
Matriz de 10x10 componentes	B	var	real
Contador	i	var	entero
Contador	j	var	entero
Número de filas de la matriz A	N	var	entero
Número de columnas de la matriz A	M	var	entero
Uno	1	cte	entero
Aspecto de resolución		Orientación para el estudiante	
Objeto	Nombre	Valor	Tipo
Entrada		Identifica qué datos se leen por teclado y cuáles ya están definidos.	
Proceso		Describe el recorrido necesario con ciclos y las condiciones o acumulaciones requeridas.	
Salida		Especifica qué se debe mostrar y en qué orden.	
Pensamiento crítico		¿Por qué la matriz traspuesta cambia filas por columnas? ¿Cómo se modifican los límites al imprimir B?	

Análisis

El algoritmo genera la matriz traspuesta intercambiando filas por columnas mientras copia los elementos de una matriz a otra.

código Pseint:

Proceso MatrizTraspuesta

Definir A, B Como Real

Definir i, j, N, M Como Entero

Dimension A[10,10]

Dimension B[10,10]

Escribir "Ingrese el numero de filas:"

Leer N

Escribir "Ingrese el numero de columnas:"

Leer M

Para i <- 0 Hasta N - 1 Hacer

Para j <- 0 Hasta M - 1 Hacer

Escribir "Ingrese un elemento:"

Leer A[i,j]

B[j,i] <- A[i,j]

FinPara

FinPara

Escribir "Matriz traspuesta:"

Para i <- 0 Hasta M - 1 Hacer

Para j <- 0 Hasta N - 1 Hacer

Escribir Sin Saltar B[i,j], " "

FinPara

Escribir ""

FinPara

FinProceso

Tabla de objetos:

Objeto	Nombre	Valor	Tipo
Matriz original	A	var	real
Matriz traspuesta	B	var	real
Contador de filas	i	var	entero

Objeto	Nombre	Valor	Tipo
Contador de columnas	j	var	entero
Número de filas de A	N	var	entero
Número de columnas de A	M	var	entero
Uno	1	cte	entero

Prueba de escritorio:

Supongamos la matriz A:

1 2 3

4 5 6

Para matriz B

Operación	Resultado
-----------	-----------

$B[0,0] = A[0,0] + 1$

$B[1,0] = A[0,1] + 2$

$B[2,0] = A[0,2] + 3$

Operación	Resultado
-----------	-----------

$B[0,1] = A[1,0]$	4
-------------------	---

$B[1,1] = A[1,1]$	5
-------------------	---

$B[2,1] = A[1,2]$	6
-------------------	---

Matriz traspuesta B

1	4
---	---

2	5
---	---

3	6
---	---

código en C:

```
#include <stdio.h>
```

```
int main() {
```

```
    float A[10][10], B[10][10];
```

```
    int i, j, N, M;
```

```
    printf("Ingrese el numero de filas: ");
```

```
    scanf("%d", &N);
```

```
printf("Ingrese el numero de columnas: ");
```

```
scanf("%d", &M);
```

```
for(i = 0; i < N; i++) {
```

```
    for(j = 0; j < M; j++) {
```

```
        printf("Ingrese el elemento [%d][%d]: ", i, j);
```

```
        scanf("%f", &A[i][j]);
```

```
        B[j][i] = A[i][j];
```

```
    }
```

```
}
```

```
printf("\nMatriz traspuesta:\n");
```

```
for(i = 0; i < M; i++) {
```

```
    for(j = 0; j < N; j++) {
```

```
        printf("%.2f ", B[i][j]);
```

```
    }
```

```
    printf("\n");
```

```
}
```

```
return 0;
```

```
}
```

```
Ingrese el numero de filas: 2
Ingrese el numero de columnas: 2
Ingrese el elemento [0][0]: 1
Ingrese el elemento [0][1]: 2
Ingrese el elemento [1][0]: 5
Ingrese el elemento [1][1]: 6

Matriz traspuesta:
1.00 5.00
2.00 6.00

Process returned 0 (0x0)   execution time : 8.418 s
Press any key to continue
```

Reflexión:

Este ejercicio ayuda a comprender cómo las matrices pueden reorganizarse intercambiando filas por columnas para obtener nuevas estructuras de datos.

1. Rúbrica de evaluación: pensamiento crítico de resolución

Puntaje total: 20 puntos. Esta rúbrica valora tanto la solución técnica como la capacidad del estudiante para justificar su razonamiento.

Criterio	Excelente	Bueno	En proceso	Puntaje
Comprensión del problema	Interpreta correctamente el enunciado, identifica entradas, proceso y salida, y distingue si el ejercicio usa vector o matriz.	Comprende la idea general, aunque omite algún detalle de entrada, salida o restricción.	Presenta confusión sobre lo que debe calcularse o mostrarse.	4 pts
Identificación de objetos y tipos	Reconoce variables, constantes, contadores, acumuladores, banderas o auxiliares; asigna tipos adecuados.	Identifica la mayoría de objetos, pero con pequeños errores de tipo o función.	La tabla de objetos está incompleta o no corresponde al problema.	4 pts
Diseño lógico del algoritmo	Propone ciclos, condiciones, acumulaciones o intercambios coherentes con el problema y sus límites.	La lógica principal es adecuada, aunque puede tener errores menores de límite o recorrido.	La lógica no permite resolver correctamente el ejercicio.	5 pts
Prueba de escritorio y validación	Comprueba la solución con datos pequeños y explica cómo cambian las variables paso a paso.	Realiza una prueba parcial, pero con poca explicación de los cambios internos.	No evidencia validación o la prueba no coincide con el algoritmo.	3 pts
Justificación crítica de decisiones	Argumenta por qué usa acumulador, bandera, auxiliar, inicialización o recorrido específico; detecta riesgos como índices y tipos de dato.	Justifica algunas decisiones, aunque de manera breve o incompleta.	No justifica sus decisiones o solo copia código sin explicar.	3 pts
Presentación y orden	Entrega clara, ordenada y fácil de revisar, con nombres de variables legibles y comentarios útiles.	La entrega es entendible, pero puede mejorar en organización o comentarios.	La presentación dificulta la revisión del ejercicio.	1 pt